

Procedural settlements for DnDRom

Findings and recommendation

DnDRom should generate a settlement as a connected spatial design: its reason for existing, terrain relationship, public spaces, circulation, districts, parcels, architecture and usable interiors. Building count alone is a poor description of a town. A harbor must have a working waterfront; a market town needs a legible center; a castle needs defensible approaches and connected internal spaces. The recommendation is a hybrid procedural system, directed by local AI and governed by measured geometry.

Published village research combines terrain suitability, incremental settlement growth, road development and slope-aware parcel formation. Street-modeling research provides controls for regular, radial and terrain-following networks. Building shape grammars provide a separate mechanism for generating architecture from the resulting footprints. These techniques address different levels of the problem and should be combined rather than expecting one algorithm to solve everything.^{1, 2, 4}

The current implementation explains the repeated-grid appearance. `proposeLocationProgram` in `apps/desktop/src/domain/worldPipeline.ts` samples 220 candidate positions in fixed rows and lanes. Most buildings are 9×12 m, warehouses are 12×12 m, orientation is one of two opposite headings, and height cycles through three values. Roads connect accepted buildings afterward. The default `architectureRecipe` emits the same frame, roof and doorstep; building role mostly changes market openness. These are verified source observations, not a renderer or AI inference.

The existing foundations are useful. Accepted building IDs, recipes, entrances and transport are already persisted, and `sharedWorldScene.ts` derives local and distant entities from those plans. The proposal retains that ownership model while replacing the settlement solver and expanding its architecture contract. The separate `generateCgaBuilding` helper already has some footprint and façade variation, but it is not the shared-world default recipe responsible for this town.

The settlement upgrade should be accepted on three independent grounds: spatial validity, visual quality and runtime cost. A town can be connected and still look like a barracks; it can look impressive and still have inaccessible doors. Reference comparisons and actual packaged traversal are therefore release requirements alongside automated validation.

This report is a researched implementation design. It does not change the 0.3.3 executable or claim that the replacement settlement system is implemented. The broader population, economy, voiced DM and daily-activity systems remain a later phase. The intended target remains local operation with 8 GB VRAM and 16 GB RAM; performance must be measured with the local model active.

1. Emilien et al., 2012. [Procedural Generation of Villages on Arbitrary Terrains](#).

2. Chen et al., 2008. [Interactive Procedural Street Modeling](#).

4. Müller et al., 2006. [Procedural Modeling of Buildings](#).

Algorithm selection

Method	Useful contribution	Recommended boundary
Terrain-aware incremental growth	Paths and homes influence each other; terrain changes the settlement shape	Village and fringe growth, with bounded iterations
Direction fields and street graphs	Blend shoreline, contour, radial and planned alignments	City streets and district structure, followed by geometric validation
Frontage-aware parcel subdivision	Gives buildings usable street access and controllable lot proportions	Town blocks; use irregular rural parcels where blocks do not exist
Context-sensitive shape grammar	Generates massing, floors, façades and detailed architectural parts	Main building compiler, with style-specific production rules
Room-program optimization	Converts required spaces and relationships into internal layouts	Playable buildings within accepted exterior envelopes
Wave Function Collapse	Enforces local module compatibility	Façade bays, roof components and decoration after global constraints

Emilien and colleagues model small European villages with interest maps, coupled road/settlement growth and anisotropic parcel expansion. Its strongest relevance is adapting settlements to terrain; its regional scope does not justify treating its examples as a universal town model. Chen and colleagues guide street graphs with editable tensor fields. A field suggests directions; it does not itself guarantee feasible slopes, useful intersections or access to every building.^{1, 2}

Vanegas and colleagues explicitly address parcel subdivision and preserving parcel correspondence across edits. That matters for a VTT because a building may already contain player changes and quest associations. Müller and colleagues demonstrate context-sensitive building grammars, while Merrell and colleagues generate floor plans from architectural programs. Exterior and interior generation should share constraints even when their meshes load at different times.^{3, 4, 5}

Wave Function Collapse supports local pattern constraints and may reach contradictions. DnDRom should use it only where local compatibility is the right problem. Global road reachability, castle access and ferry connectivity still require graph and geometric checks. Random scattering and simple polygon subdivision remain useful utilities, but neither supplies a complete settlement design.⁶

Recommended composition: reserve immutable geographic and transport anchors; sketch the main routes and public spaces; grow districts or small settlement clusters; solve parcels; create role-specific building programs; compile architecture; validate the entire result. Allow bounded feedback between parcel capacity and secondary streets. Avoid locking every road before discovering that the required buildings cannot fit, and avoid placing every building before discovering that useful streets cannot fit.

An adaptation matters: the village paper permits suppressing seeds that lose road access and acknowledges substantial parameter tuning. DnDRom must instead repair or reject a plan when that would remove a required building.¹

1. Emilien et al., 2012. [Procedural Generation of Villages on Arbitrary Terrains](#).

2. Chen et al., 2008. [Interactive Procedural Street Modeling](#).

3. Vanegas et al., 2012. [Procedural Generation of Parcels in Urban Modeling](#).

4. Müller et al., 2006. [Procedural Modeling of Buildings](#).

5. Merrell et al., 2010. [Computer-Generated Residential Building Layouts](#).

6. Gumin, living repository. [WaveFunctionCollapse](#).

Terrain, anchors and streets

The following is a proposed DnDRom algorithm. Begin with the accepted world's eroded terrain, hydrology, coastline, existing routes and authored edits. Build a local analysis field for dry ground, slope, drainage, buildable area and access costs. Terrain selection and settlement composition should operate on the same samples used by near and distant rendering. A coastal label alone is insufficient: the solver must find a usable waterfront and berth.

Reserve anchors before housing. A harbor gets a dock approach, cargo handling space and an inland connection; a castle gets a gate approach and its principal enclosure; a market town gets a square and regional road junction. Attach each anchor to an explicit reason and required connection. Landmark position is a spatial constraint, not a random decoration placed at the end.

Use a route hierarchy: regional approaches, main streets, residential/service lanes and pedestrian passages. Direction fields can combine shoreline tangents, terrain contours and selected district alignments. The published tensor-field approach supports combining these influences, but DnDRom should restrict orthogonal patterns to places where the settlement program calls for them.²

Score candidate paths using distance, grade, curvature, cut/fill, water crossings and damage to protected parcels. Grade limits and passage widths are game-design parameters, not claimed historical constants. Separate cart-capable routes from steps and steep footpaths so a mountain town can remain connected without pretending every alley is a wagon road. Prefer shared trunks and useful loops over repeated parallel paths to adjacent doors.

Maintain an explicit graph of intersections, centerlines and route categories. Snap near intersections, remove microscopic edges, reject accidental overlaps and distinguish an at-grade crossing from a bridge. After smoothing, resample the route against terrain and clearance constraints: a spline that looks smooth can cut through a house or violate the original slope limit.

Construct the road surface from its graded longitudinal profile and a controlled cross-section. Do not inherit the hillside's sideways roll. Stepped terraces and retaining structures should be recorded as bounded terrain edits, with their water and drainage consequences checked. Reserve plaza and quay surfaces as polygons connected to the road graph, allowing buildings to face actual public space rather than isolated line segments.

Early output should expose the settlement skeleton: main routes, public spaces, landmark masses and coarse building envelopes. This is enough to review the settlement's shape before expensive façades and small details are compiled.

2. Chen et al., 2008. [Interactive Procedural Street Modeling](#).

Districts, parcels and growth history

A settlement should support several spatial forms. A fishing hamlet may follow one shore path. A market town may gather around a square and several approaches. A city may contain a planned quarter beside older irregular districts. A hill settlement may occupy successive terraces. These are composable generation policies, not fixed map templates.

For dense districts, derive blocks from the street graph and subtract streets, waterways, public spaces and protected areas. Divide buildable blocks into frontage strips, then subdivide those strips into parcels with role-appropriate width, depth and coverage. For loose villages, use terrain-aware seed growth and parcel expansion instead. Preserve leftovers as gardens, service yards, passages or undeveloped land; do not force a building into every sliver. Parcel research offers evidence that subdivision can be controllable and preserve correspondence through editing.³

Proposed parcel records should contain a polygon, frontage edge, primary access point, secondary access where needed, buildable envelope, slope/foundation policy, allowed building families and protected open space. One plot may hold a main house plus workshop and yard; a temple or castle complex may own multiple connected parcels. A building's footprint is therefore not interchangeable with its parcel.

Select orientation from the frontage or courtyard relationship. Sample variety conditionally: a warehouse needs cargo access, a street shop benefits from a public frontage, and a household courtyard requires a coherent entrance sequence. Solve the footprint and entrance together, then validate roof overhangs, balconies and stairs against neighboring plots. Shared walls need explicit permission and compatible geometry.

Create a short synthetic growth history as a design tool: initial landing or stronghold, first route, market expansion, denser infill and a later planned extension. Preserve each phase's alignments and material tendencies. This can create readable age differences without running a population simulation. Chronology is persisted metadata that future activity systems can use, not a claim that historical urban development has been simulated.

Use bounded revisions when capacity fails: change a secondary lane, merge suitable parcels, expand into approved land or test another unaccepted site. Mandatory roles must never disappear silently. Existing accepted terrain and player edits are fixed constraints. A geometric solver should return the actual cause of failure—insufficient frontage, steep access, flood exposure or incompatible footprint—so the planner can make a meaningful revision.

3. Vanegas et al., 2012. [Procedural Generation of Parcels in Urban Modeling](#).

Building architecture and visible variation

A building generator needs an architectural program, not only width, depth and color. The proposed program identifies role, occupied floors, required rooms, access, public frontage, storage/service needs, wealth, construction system and building age. The compiler then produces a massing arrangement that fits the parcel: a narrow street house, courtyard compound, hall with wings, workshop with yard, warehouse with loading frontage or tower with attached service ranges.

CGA research separates mass modeling from façade and detail rules, with context checks for interactions such as doors and adjacent walls. DnDRom should adopt that hierarchy rather than replicating a complete house as the basic repeated module. Room-layout research further supports planning internal relationships before committing the complete shell.^{4, 5}

Proposed structural operators include footprint union and setback, extrusion by storey, floor/ceiling insertion, façade subdivision, real door/window openings, roof generation, porch/arcade construction, stairs, towers and courtyard enclosure. Extend roof generation beyond one ridge: hips, cross-gables, lean-tos, parapets, curved eaves and connected roof masses. Arbitrary concave roofs need a robust geometric solution and drainage checks; intersecting roof wedges are not a general solution.

Give each role a recognizable silhouette and circulation pattern. A warehouse may have fewer larger openings, a tall loading bay and a crane attachment. An inn may combine a larger public ground floor with rooms above and a yard entrance. A shrine may have a distinctive forecourt or roof treatment. These are proposed fantasy design rules, selected through regional architecture definitions rather than hard-coded cultural stereotypes.

Variation should occur at several scales: district height and density, parcel width, massing and roof topology, façade rhythm, entrances and appendages, then materials and wear. Neighboring buildings share a regional vocabulary while differing within it. Use stable building seeds and a local similarity penalty to avoid long runs of identical silhouettes, except where intentional repetition such as barracks or planned terraces is requested.

Material quality remains a separate art requirement. Maintain coherent texel density, trim placement, roof scale, corner treatment and physically consistent roughness/normal maps. Distribute weathering through exposure, runoff, ground contact and repairs rather than a uniform random overlay. Structural defects should be visible first in neutral lighting; excellent textures cannot rescue a uniformly repeated mass model.

Persist the generated building program and selected grammar production. Use the same exterior footprint, floor elevations, entrances and building ID for interior construction. Resolve essential stairs and door clearances before accepting the exterior. Meshes and movable furnishing can load later, while the required internal access graph already exists.

4. Müller et al., 2006. [Procedural Modeling of Buildings](#).

5. Merrell et al., 2010. [Computer-Generated Residential Building Layouts](#).

Regional styles and fantasy extensions

Style must affect settlement morphology and construction, not simply replace a roof texture. A proposed StyleProfile should distinguish cultural/fictional tradition, period, available materials, climate response, structural system, plot conventions, roof families, opening proportions, public-space traditions and ornamental vocabulary. District, institution, household wealth and building age then modify that shared profile.

Real references demonstrate why a single label is inadequate. UNESCO describes Ping Yao through its walls, streets, shops, dwellings and temples across centuries of Han urban development. Xidi and Hongcun provide a different reference centered on village fabric and integrated waterways. These should inform distinct profiles, not a universal “Chinese building” with interchangeable curved roofs.^{7, 8}

Shibam offers another useful corrective: its dense mud-brick tower houses stand within a fortified rectangular street-and-square plan. Regularity itself is not a realism defect. Himeji provides a castle reference where linked building masses, layered roofs and defensive organization are integral to the architectural identity.^{9, 10}

Proposed profile family	Spatial rules to author	Building grammar emphasis
Timber market town	Street frontages, rear yards, a market center	Narrow houses, projecting floors, shopfronts and mixed roof masses
Southern Anhui-inspired village	Waterway relationships, lanes and compounds	Courtyard forms and a regionally reviewed wall/roof vocabulary
Ping Yao-inspired walled town	Gates, ordered routes, commercial frontages	Varied dwelling compounds, shops and civic/religious buildings
Hadrami-inspired tower settlement	Dense parcels, squares and defensive boundary	Tall earthen masses, vertical organization and graded openings
Japanese castle complex	Layered enclosures and controlled approaches	Distinct keeps, connecting structures and layered roof compositions
Invented mountain or forest culture	Explicit access, support and climate rules	New arch, bridge, platform, carved or branching operators

The table is an implementation proposal, not an exhaustive historical classification. Each real-world-inspired profile needs a focused reference set for its region and era. Fantasy profiles can change physical assumptions, but must state them: floating platforms still need traversal connections; tree dwellings need support and access; underground halls need three-dimensional spatial ownership.

Reference images should guide observable features such as massing, roof pitch, materials and opening rhythm. Hidden interiors, structural systems and geographic context cannot be uniquely recovered from one image. Let the local model propose an editable profile with uncertainty recorded. Cosmetic restyling preserves footprints and entrances; changes to courtyards, floor count or circulation require an explicit structural revision.

7. UNESCO, property 812. [Ancient City of Ping Yao](#).
8. UNESCO, property 1002. [Ancient Villages in Southern Anhui – Xidi and Hongcun](#).
9. UNESCO, property 192. [Old Walled City of Shibam](#).
10. UNESCO, property 661. [Himeji-jo](#).

Castles and fortified settlements

Castles need their own compound-level generator. A keep is one building; a castle is a relationship between terrain, access, enclosure, accommodation, service spaces and defense. English Heritage's Warkworth description documents a naturally defended river-loop setting, a great tower, bailey buildings, a curtain wall and gatehouse. Its changing ranges and internal rooms show why a rectangular wall with repeated corner towers is an inadequate universal model.¹¹

Dusterwald's terrain-adaptive castle thesis is a direct procedural precedent, but it is a voxel-world research prototype rather than a ready-made photorealistic UE runtime. Himeji supplies a separate historical example of compound organization, roof layering and connected defensive structures. Neither reference supports a single castle grammar for all places and periods.^{12, 10}

Proposed generation begins with a defensible area and reachable gate approach. Select a family: motte-and-bailey, compact tower compound, courtyard stronghold, concentric fortress, ridge castle, palace-fortress or a clearly fictional derivative. Fit outer and inner enclosure polygons to terrain, allowing asymmetry and preserving circulation space. Towers follow vulnerable segments, corners and gate needs rather than equal spacing alone.

Build a compound graph: approach → gate → outer court → controlled inner entry → inner court or keep. Service ranges attach to suitable courts. Add a hall, kitchen, storage, water source, accommodation and optional sacred or administrative spaces according to the requested program. Keep these as functional spaces with access requirements, even before inhabitants exist. Wall walks, gate passages and stairs belong to the traversal graph.

Validate actual geometry along the entire defense perimeter: foundations, continuity, gate clearances, tower-to-wall joins and walkable connections. Courtyard grades must work with door thresholds. A wall may climb or step with terrain while usable floors stay level. Smoothing an enclosure polygon must not push a tower over a cliff or close a gate approach.

For play, score sightlines, distinct approaches, courtyards, elevated positions, bottlenecks and alternate routes. These are encounter-design considerations, not proof of historically optimal fortification. A ruined castle should first be generated as a coherent intact compound, then receive a persistent damage pattern that removes structural portions and updates collision/access. Random holes should not accidentally erase every route to the required gameplay space.

The compound ID owns enclosure, court and building IDs. Distant geometry retains the same wall outline, principal towers and keep silhouette. Interior and wall-walk detail can stream later without substituting a different castle when the players arrive.

10. UNESCO, property 661. [Himeji-jo](#).

11. English Heritage, undated. [Description of Warkworth Castle and Hermitage](#).

12. Dusterwald, 2015. [Procedural Generation of Voxel Worlds with Castles](#).

Harbor-town reference target

The first acceptance scene should reuse the existing island harbor and ferry relationship. The following is a proposed design brief for that fixture, not a claim of a newly generated scene. Retain the accepted water and surrounding terrain, then create a new settlement revision with enough usable waterfront and land for the full program.

Keep the agreed 36-building default: 24 homes, eight working/storage buildings, and four public/commercial buildings. The eight work buildings can be four warehouses plus a boat-repair workshop, net-maker, cooper and smithy. The four public/commercial buildings can be an inn, covered market, town/customs hall and shrine. These particular assignments are fantasy program choices, not historical ratios. Docks, cargo yards and an open market square are additional spatial reservations rather than hidden substitutes for required buildings.

Place storage and boat-related work near verified cargo or waterside access. Connect the landing to the market and inland lanes. Arrange homes in several frontage clusters with varied plot depths and a few shared yards, preserving paths to the waterfront. Let the hall or shrine establish a modest landmark, and give the inn a larger public frontage. Suitable shoreline segments may remain a natural beach; a harbor need not turn its entire coast into a quay.



The diagram is a schematic relationship sketch, not a measured site plan or engine output. The solver must determine actual orientation, dimensions and buildability. The ferry route and matching mainland landing remain authoritative transport edges in the same world.

Require a useful variety of massing families in this fixture—for example narrow homes, courtyard or winged homes, long warehouses, open workshops, a larger inn and distinctive public buildings. The exact mix should be selected from the regional profile. Review the town both from the water and at street level; variation visible only from a top-down editor view is insufficient.

Static place-making comes after access validation: cargo areas, fences, well courts, shop thresholds, workshop yards and modest vegetation. Populate these from building role and usable space. Preserve reserved hooks for a later living-world system, including berth positions, shop counters, work areas and household entrances, without implementing NPC simulation in this upgrade.

Local AI and persistent plans

The local model should produce a settlement brief and architectural intentions, while deterministic solvers own exact geometry. The brief includes settlement purpose, scale, morphology preferences, district relationships, landmarks, required roles, style references and unusual constraints. An optional local vision model can propose style attributes from images. The application must expose unsupported structural requirements instead of quietly mapping every unfamiliar request to the same house.

CityGenAgent, a 2026 preprint, separates block and building programs and trains models to improve program validity and visual alignment. It is relevant evidence for hierarchical language-to-procedure design. It is not evidence that an unchanged small local model will instantly produce production-quality settlements: the paper discusses significant generation time and constrained-device deployment limitations. Its published training and evaluation setup should not be treated as DnDRom's runtime budget.¹⁹

Proposed bounded operations are `describe_site`, `propose_program`, `query_capacity`, `propose_districts`, `select_style_rules`, `validate_plan` and `revise_plan`. Outputs are typed data or a bounded grammar syntax tree. Evaluate geometry, collisions and access in engine code. Validate a proposed grammar's expansion depth, part count, material references and bounds before accepting it; unrestricted generated code is unnecessary for this workflow.

Extend the persistent settlement contract with `SettlementProgram`, `DistrictPlan`, `StreetGraph`, `ParcelPlan`, `BuildingProgram`, `CompoundPlan` and `StyleProfile`. Each accepted record has a stable ID and generator version. Store authored decisions separately from regenerable geometry. Use semantic parent IDs and persisted allocation IDs; avoid array-index identity that shifts when a new building role is inserted earlier in a list.

Track dependencies so a roof restyle invalidates roof/material output, while a revised street may invalidate affected parcels and buildings. Accept revisions transactionally after validation. Preserve untouched buildings, terrain, entrances and user edits. Cache eviction or camera motion must never trigger a new layout decision.

The interface should show planning stages and usable results: site measured, routes established, parcels solved, buildings compiled, local area ready, distant refinement. Show unmet constraints plainly and retain the last accepted plan. Bounded repair attempts can adjust pending work, while a change to accepted geography creates a reviewable revision. Once accepted, the plan is sufficient for reconstruction even when the model is unavailable.

19. Liu et al., 2026. [Imagine a City: CityGenAgent for Procedural 3D City Generation](#).

Unreal rendering and local runtime

Epic's UE 5.8 City Sample PCG is a particularly relevant reference: it builds a city through dependent graphs for districts, roads, lots and buildings, using internal Unreal systems rather than the original sample's external Houdini workflow. Its shape definitions arrange modules along splines and produce exclusion footprints. The sample remains an authoring/learning example; its documentation describes multi-minute regeneration and a world without the original traffic/pedestrian simulation. Its separate background silhouettes should not replace DnDRom's shared near/far geography.¹³

The installed UE 5.8 checkout contains PCG and PCG Primitives, but the latter's .uplugin marks it experimental. DnDRom's project currently uses its own native scene importer and does not explicitly integrate PCG in its game-module dependencies. PCG runtime generation is documented for standalone builds; that does not prove every sample graph, asset-creation operation or editor MCP tool works inside a packaged game.¹⁴

Recommended integration: keep the world manifest and settlement solver authoritative. First build one packaged PCG adapter experiment consuming an accepted parcel/building plan and spawning cooked modules. Compare it with the existing native instancing path. Adopt PCG where it improves delivery, with no requirement for end users to run the Unreal Editor, Houdini or a cloud service. PCG grammar support is useful for repeated architectural sequences, but must be paired with the massing and access solver.¹⁵

Use shared modules for windows, doors, beams, roof sections and arches; combine them through many distinct building programs. Epic documents instancing and per-instance custom data for reducing actor/material overhead. Choose ISM versus HISM through measurements on the relevant static, moving and Nanite-backed content rather than assuming one is universally superior.¹⁷

Do not assume arbitrary runtime-generated meshes automatically gain Nanite. Epic's Geometry Scripting guide explicitly lists limitations for DynamicMeshComponent, including Nanite, LODs and instancing, and notes game-thread execution of Blueprint calls. DnDRom currently builds native static meshes and instances; that is a different path, which also requires explicit feature verification. Prefer cooked Nanite-capable modules where useful, and maintain measured conventional representations for custom generated pieces.^{16, 18}

Derive district proxies, building silhouettes and detailed meshes from the same accepted plan. Preserve landmark identity and entrances across detail changes. Stream local terrain and useful building detail first, retain covering geometry through handoffs, and advance the fixed-center reveal only through ready coverage. Batch imports, bound worker queues and track GPU memory, upload work and collision-building time separately. Dense decorative modules and interiors should never block first local play.

13. Epic Games, UE 5.8 documentation. [City Sample PCG](#).

14. Epic Games, UE 5.8 documentation. [Using PCG Generation Modes](#).

15. Epic Games, UE 5.8 documentation. [Using Shape Grammar With PCG](#).

16. Epic Games, UE 5.8 documentation. [Geometry Scripting Users Guide](#).

17. Epic Games, UE 5.8 documentation. [Instanced Static Mesh Component](#).

18. Epic Games, UE 5.8 documentation. [Nanite Virtualized Geometry](#).

Implementation sequence and acceptance

Phase 1 replaces the fixed parcel lattice with anchor, street, public-space and parcel planning for the harbor fixture. Extend the versioned contracts first, preserve the old accepted layout until a new revision succeeds, and retain the accepted ferry and world geography. The output must already read as a harbor town in simple massing views before material polish proceeds.

Phase 2 adds role-driven building programs, several footprint and roof families, verified openings, stairs and usable entry sequences. Author one coherent regional style thoroughly, then a distinctly different profile to prove that style changes affect structure. Expand toward castles, terraced towns, dense cities and fantasy profiles through the same contract rather than separate unconnected scene generators.

Phase 3 integrates detail levels and validates the packaged runtime. Compare the native assembly path with the bounded PCG experiment. Profile settlement planning, architecture compilation, collision creation, first visible local coverage, first playable space, distant completion and steady-state movement independently. Preserve the wave system and existing fog/coverage regressions.

Acceptance area	Required evidence
Settlement function	Required roles and counts exist; harbor cargo and ferry access work
Geometry	Valid parcel polygons; no unintended building/road/water overlaps; supported foundations
Traversal	Entrances connect to streets/courts; required floors and castle wall walks are reachable
Variation	Distinct role silhouettes, parcel shapes and roof compositions; no accidental repeated-grid default
Style	Two structurally different profiles remain coherent at street, town and horizon scales
Persistence	Save/load, cache eviction, scene switching and added requests preserve accepted IDs and edits
Rendering	Near/far footprints agree; no gaps, duplicate surfaces or disappearance during camera movement
Runtime	Cold and cached timings, p50/p95/p99 frame times, peak RAM/VRAM and active-model contention

Use a deterministic fixture matrix: flat and sloped harbors, a ridge castle, a terraced town, a courtyard settlement, a dense walled city, and an impossible site. Run a broad seed set for geometry and connectivity, plus fixed reference seeds for visual review. For intentionally ordered settlements, repeated alignments are acceptable; tests should detect an unintended default rather than penalize every grid.

Maintain view-based visual checks from the harbor approach, market, residential lane, castle gate and distant horizon. Compare roof/footprint repetition, silhouette hierarchy, street enclosure, landmark visibility and terrain integration. These measures support human review rather than replacing it with a single aesthetic score. Present a before/after comparison against the current harbor and report any unresolved defects.

Release only after the new settlement fixture meets those gates. The 1080p/30 FPS target on 8 GB VRAM and 16 GB RAM with local AI active remains an unverified target. This research does not establish a reliable generation-time promise or that the full City Sample can meet that hardware budget.

Sources

The following primary research, official documentation and heritage references support the design. Algorithm publications establish precedents; engineering choices and fixture thresholds in this report are DnDRom proposals. Dynamic documentation was reviewed on 9 September 2026. Dates below refer to the work or documented edition, not search-engine crawl dates.

1. Arnaud Emilien, Adrien Bernhardt, Adrien Peytavie, Marie-Paule Cani and Eric Galin. [Procedural Generation of Villages on Arbitrary Terrains](#). The Visual Computer 28, 809–818, 2012. Terrain suitability, coupled village/road growth, anisotropic parcels and slope-aware architecture; European-village scope. Full paper accessed; method and limitation sections examined.
2. Guoning Chen, Gregory Esch, Peter Wonka, Pascal Müller and Eugene Zhang. [Interactive Procedural Street Modeling](#). ACM Transactions on Graphics 27(3), 2008. Direction fields, terrain/water boundaries and street-graph editing.
3. Carlos A. Vanegas, Tom Kelly, Basil Weber, Jan Halatsch, Daniel G. Aliaga and Pascal Müller. [Procedural Generation of Parcels in Urban Modeling](#). Computer Graphics Forum 31, 681–690, 2012. Publisher abstract supports controllable subdivision and persistent parcel correspondence; detailed implementation is not reproduced here.
4. Pascal Müller, Peter Wonka, Simon Haegler, Andreas Ulmer and Luc Van Gool. [Procedural Modeling of Buildings](#). ACM Transactions on Graphics 25(3), 614–623, 2006. Context-sensitive building shape grammar, massing and façade construction. Full paper accessed; overview and context-query sections examined.
5. Paul Merrell, Eric Schkufza and Vladlen Koltun. [Computer-Generated Residential Building Layouts](#). ACM Transactions on Graphics 29(6), 2010. Architectural programs and optimized floor plans; residential evidence, not universal castle/interior validation.
6. Maxim Gumin. [WaveFunctionCollapse](#). Original project documentation, begun 2016; living repository. Local pattern compatibility and contradictions. Used as an algorithm reference, not as a tested integration.
7. UNESCO World Heritage Centre. [Ancient City of Ping Yao](#). Official property description; inscribed 1997. A geographically and historically specific reference for walled urban fabric and varied building roles.
8. UNESCO World Heritage Centre. [Ancient Villages in Southern Anhui – Xidi and Hongcun](#). Official property description; inscribed 2000. Village plans and integrated water systems.
9. UNESCO World Heritage Centre. [Old Walled City of Shibam](#). Official property description; inscribed 1982. Tower-house construction, density and an ordered street/square plan.
10. UNESCO World Heritage Centre. [Himeji-jo](#). Official property description; inscribed 1993. Japanese castle organization, linked masses and layered roofs. The page has inconsistent building counts in its summary and synthesis; no count is used in this report.

Sources and implementation evidence

11. English Heritage. [Description of Warkworth Castle and Hermitage](#). Undated official site description. Terrain setting, compound organization, internal uses and multiple construction phases.
12. Sebastian Dusterwald. [Procedural Generation of Voxel Worlds with Castles](#). University of Waikato MSc thesis, 2015. Repository abstract consulted for terrain-adaptive castle positioning and layout; performance and detailed implementation not independently evaluated.
13. Epic Games. [City Sample PCG](#). Unreal Engine 5.8 documentation, 2026 demonstration. Internal PCG city pipeline, shape definitions, authoring dependencies and sample limitations.
14. Epic Games. [Using PCG Generation Modes](#). Unreal Engine 5.8 documentation. Standalone runtime generation, hierarchy, scheduling, pooling and cleanup radii.
15. Epic Games. [Using Shape Grammar With PCG](#). Unreal Engine 5.8 documentation. Symbols, module selection and procedural sequence construction.
16. Epic Games. [Geometry Scripting Users Guide](#). Unreal Engine 5.8 documentation. Dynamic-mesh component limitations, editor-only operations and Blueprint execution constraints.
17. Epic Games. [Instanced Static Mesh Component](#). Unreal Engine 5.8 documentation. Instance grouping, ISM/HISM tradeoffs and per-instance custom data.
18. Epic Games. [Nanite Virtualized Geometry](#). Unreal Engine 5.8 documentation. Nanite asset/rendering context; not evidence of arbitrary runtime mesh conversion in DnDRom.
19. Zishan Liu and colleagues. [Imagine a City: CityGenAgent for Procedural 3D City Generation](#). arXiv:2602.05362v2, revised 27 February 2026. Hierarchical block/building programs; research evidence with explicit generation-time and constrained-device limitations.

Supplemental implementation reference: Watabou's [TownGeneratorOS](#) is the author's public medieval-city source. Its README states that it lacks some later features; it should not be treated as the complete current generator. It is useful for studying a functioning town tool, not a substitute for the terrain, interior and native-rendering requirements above.

Local source observations: `worldPipeline.ts` (default program, `architectureRecipe`, `proposeLocationProgram`); `sharedWorld.ts` (building, settlement and regional-style records); `sharedWorldScene.ts` (accepted recipes shared between views); `worldArchitecture.ts` (separate CGA-style helper); `DnDRomSceneActor.cpp` (native instanced rendering); `DnDRom.Build.cs` and `DnDRom.uproject` (current engine integration). Local UE evidence: `Engine/Plugins/Experimental/PCGPrimitives/PCGPrimitives.uplugin`, specifically `IsExperimentalVersion: true`. These are checkout observations, not external-source claims.